

User Requirements

Emp/o is a third-party e-commerce platform where merchants can sell products, shoppers can buy them, and advertisers can promote selected products. The platform also includes moderators and administrators responsible for maintaining service quality and enforcing platform rules. Emp/o itself does not own or ship products. Payments are processed through Stripe, which creates transactions records stored in the system.

Shoppers

Shoppers can browse available products using the search feature or by exploring the main and listing pages. Each product includes a title, price, image, and description. Shoppers can either buy

a product directly or add it to their cart for later checkout. The system prevents purchases that exceed available stock and notifies users if a product is unavailable.

When a purchase is made, Stripe processes the payment and returns confirmation to Emp/o.

The

corresponding transaction details are stored in the Transaction class, and the relevant merchant is

informed to deliver the product to the shopper's shipping address. Shoppers can update their profile

information and report products that appear fraudulent or inappropriate. Reported products are

reviewed by merchant moderators.

Merchants

Merchants manage their inventory through a dedicated panel that allows them to add, edit, or remove products. Each listed item includes details such as title, price, description, and image.

Merchants can also update their business information, including warehouse location and bank account details. Once a shopper purchases a product, the merchant receives notification and fulfills

the delivery independently.

Advertisers

Advertisers create campaigns that promote selected products on shopper pages. When creating a

campaign, the advertiser chooses its name and adds one or more products. Each product added to

a campaign requires payment of a fixed advertisement fee. Payment is processed through Stripe,

which generates a transaction link and stores the transaction information in the system.

Advertisers can later add more products to an existing campaign by paying additional advertisement fees through new transactions linked to the original one. Campaigns can be viewed

and managed from the advertiser's panel, where advertisers may update campaign details or

remove included products.

Moderators and Administrators

Emp/o has two types of moderators: Support Moderators and Merchant Moderators.

Support Moderators handle user issues and manage support tickets, closing them when resolved.

Merchant Moderators oversee reported products and can remove items or restrict merchant accounts if they violate platform rules.

Each moderator type includes two permission levels:

Regular Moderators handle standard reports and support requests.

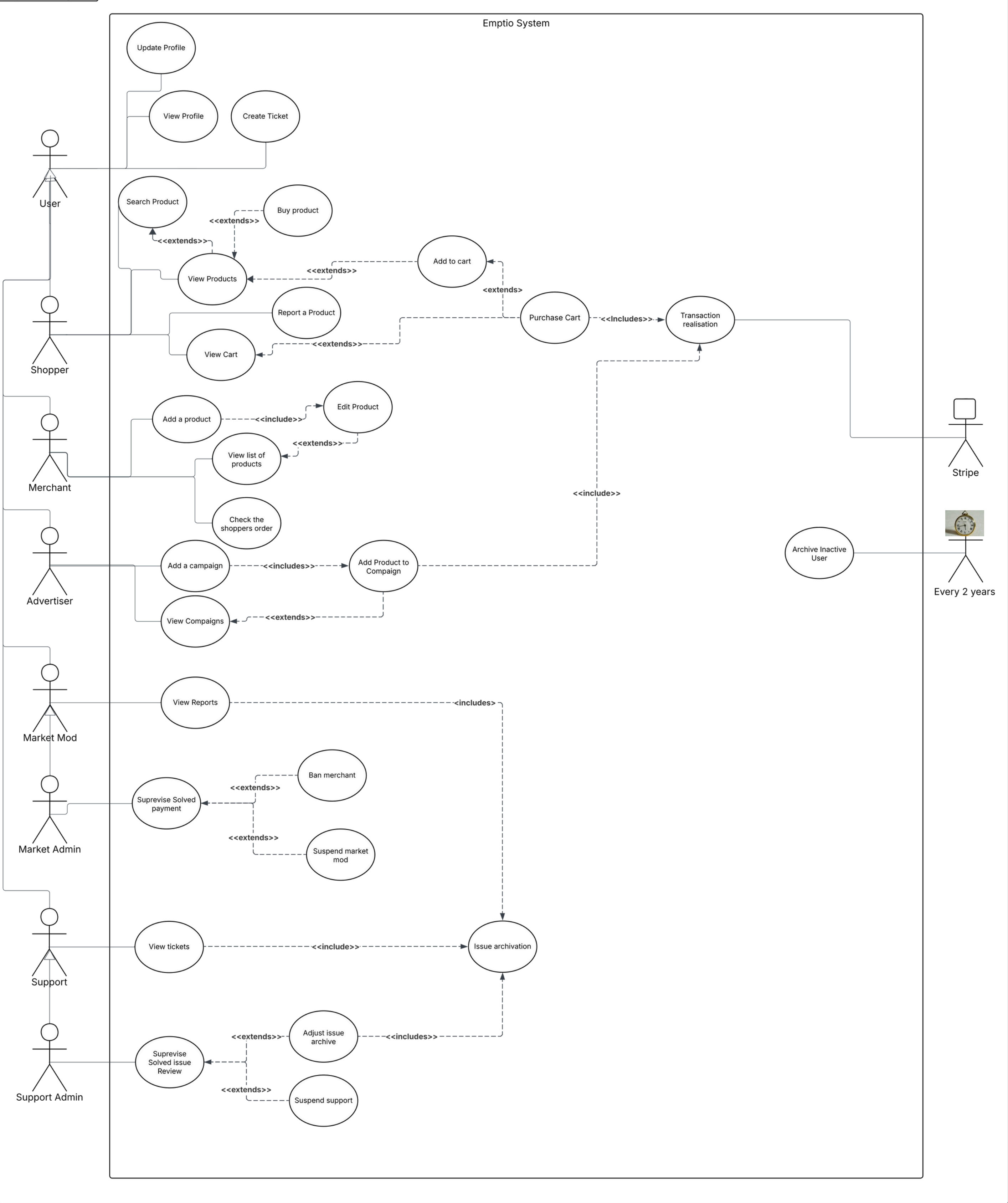
Admin Moderators serve as department heads who supervise their teams and can suspend members of their department.

Advertising and Display System

Products are displayed on shopper pages in a grid layout. Advertised products from active campaigns receive more visible placement, while regular items are shown according to availability

and category. The display logic ensures a fair mix of sponsored and regular listings while maintaining the platform's usability.

Emptio UC diagram



Use Case Scenario -Add Product

Actor: Merchant

Purpose and Context: This use case allows the merchant to add new products to their inventory on Emptio. It enables merchants to expand their online catalog by entering product details such as name, description, price, category, quantity, and images.

Assuptions:

1. The Merchant has all the required product information (name, price, description, stock quantity).

Pre-Conditions:

1. The Merchant's account is verified and active

Initiating Business Event(Trigger): 1. The Merchant clicks on the **"Add Product"** option in the merchant panel.

Basic Flow of Events:

- 1.The Merchact press on the **"Add Product"** option in the merchant panel
- 2.The System displays the Edit Product page with a form to input product details
- 3.The Merchant enters all required product information - name, price, category, description, quantity, and upload images
- 4.The System validates the entered data
- 5.The Merchant clicks **"Submit"** button
6. The System saves the product details in the database and adds the product to the merchant's list
- 7.The System displays success message - "Product added successfully"
- 8.The Merchant views the new product in inventory list

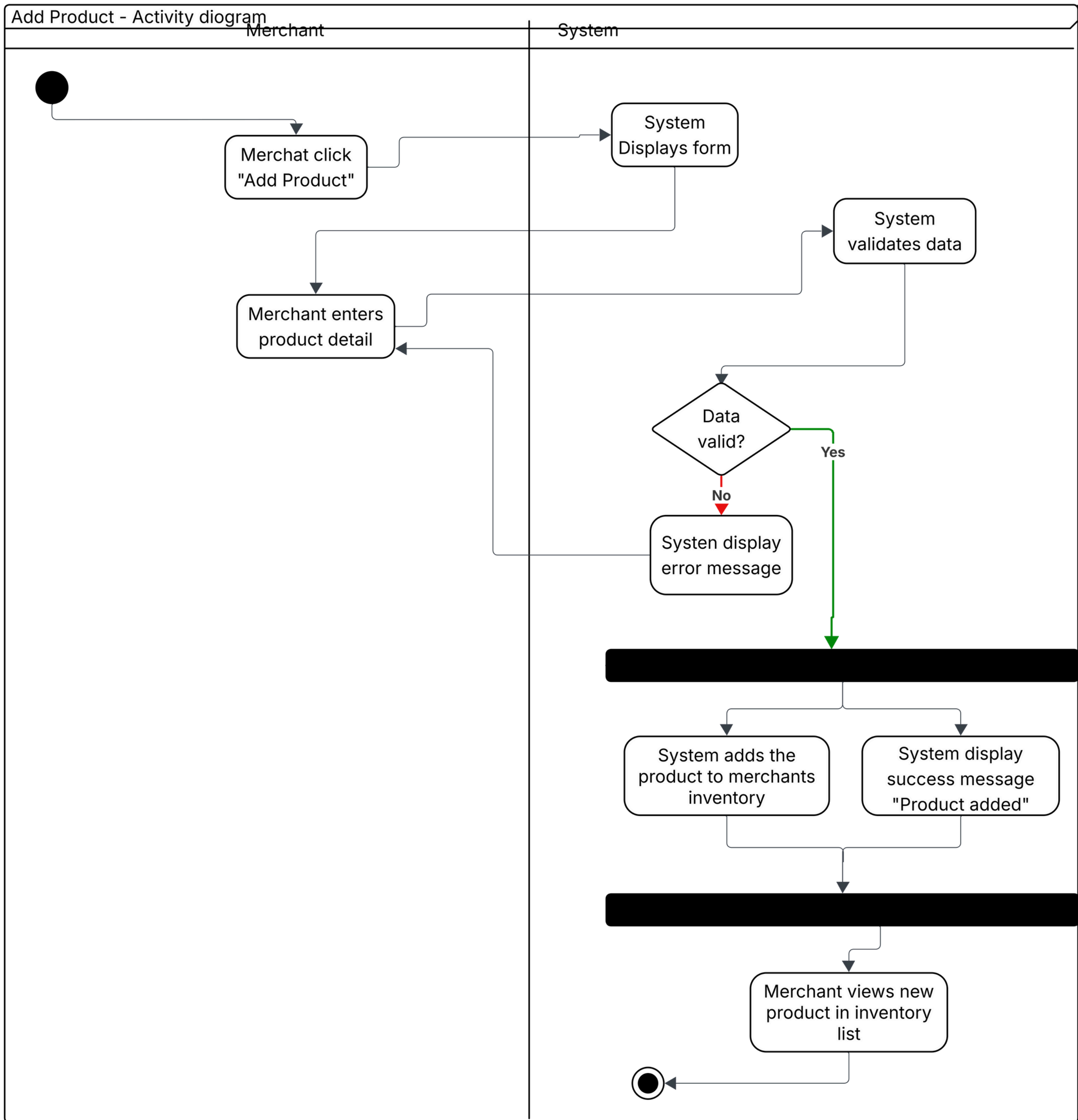
Alternative Flow of Events:

Missing Required Fields:

- 4A1.The System detects that one or more required fields are empty or contain invalid values
- 4A2.The System display an error message specifying which fields must be corrected.
- 4A3.The Merchant corrects the errors and resubmits the form

Post-Condition:

- 1.The new product is successfully added to the merchant's inventory and becomes visible in Inventory page
- 2.The system updates the product database with new entry



Use Case Scenario - Add Campaign

Actor: Advertiser

Purpose and Context: The use case is used by the advertiser to create a new advertising campaign. It allows the advertiser to define campaign details such as name, description, duration, and linked products, so that the campaign can be activated and managed within the system.

Assumptions:

- 1.The advertiser has the necessary permissions to create new campaigns.
- 2.The products intended to be included in the campaign already exist in the system.
- 3.The campaign name chosen by the advertiser is unique.

Pre-Conditions:

- 1.The advertiser has access to the campaign management panel.
- 2.The advertiser has at least one active product available to associate with the campaign.
- 3. The system has campaign creation functionality active and accessible

Initiating Business Event(Trigger): 1.The advertiser clicks on the **"Add Campaign"** button in the campaign management panel.

Basic Flow of Events:

- 1.The advertiser clicks on the **"Add Campaign"** button to start creating a new campaign.
- 2.The system displays the campaign creation form.
- 3.The advertiser enters the campaign name, description, start date, and end date.
- 4.The system records the entered information temporarily.
- 5.The advertiser selects one or more products to associate with the campaign.
- 6.The system displays a list of available products.
- 7.The advertiser confirms the selected products.
- 8.The system validates all entered campaign details and selected products.
- 9.The advertiser clicks **"Create"** to confirm the campaign creation.
- 10.The system creates a new campaign record linked to the advertiser and selected products.
- 11.The system displays a confirmation message indicating the campaign was successfully created.

Alternative Flow of Events:

Missing required information

- 3A1. The advertiser leaves one or more required fields (e.g., campaign name or dates) empty.
- 3A2. The system displays an error message prompting the advertiser to fill in all mandatory fields.
- 3A3. The advertiser completes the missing information.
- 3A4. The flow returns to step 4 of the basic flow

Invalid data range

- 8A1. During validation, the system detects that the campaign end date is earlier than the start date.
- 8A2. The system notifies the advertiser that the date range is invalid.
- 8A3. The advertiser corrects the dates.
- 8A4. The flow returns to step 8 of the basic flow.

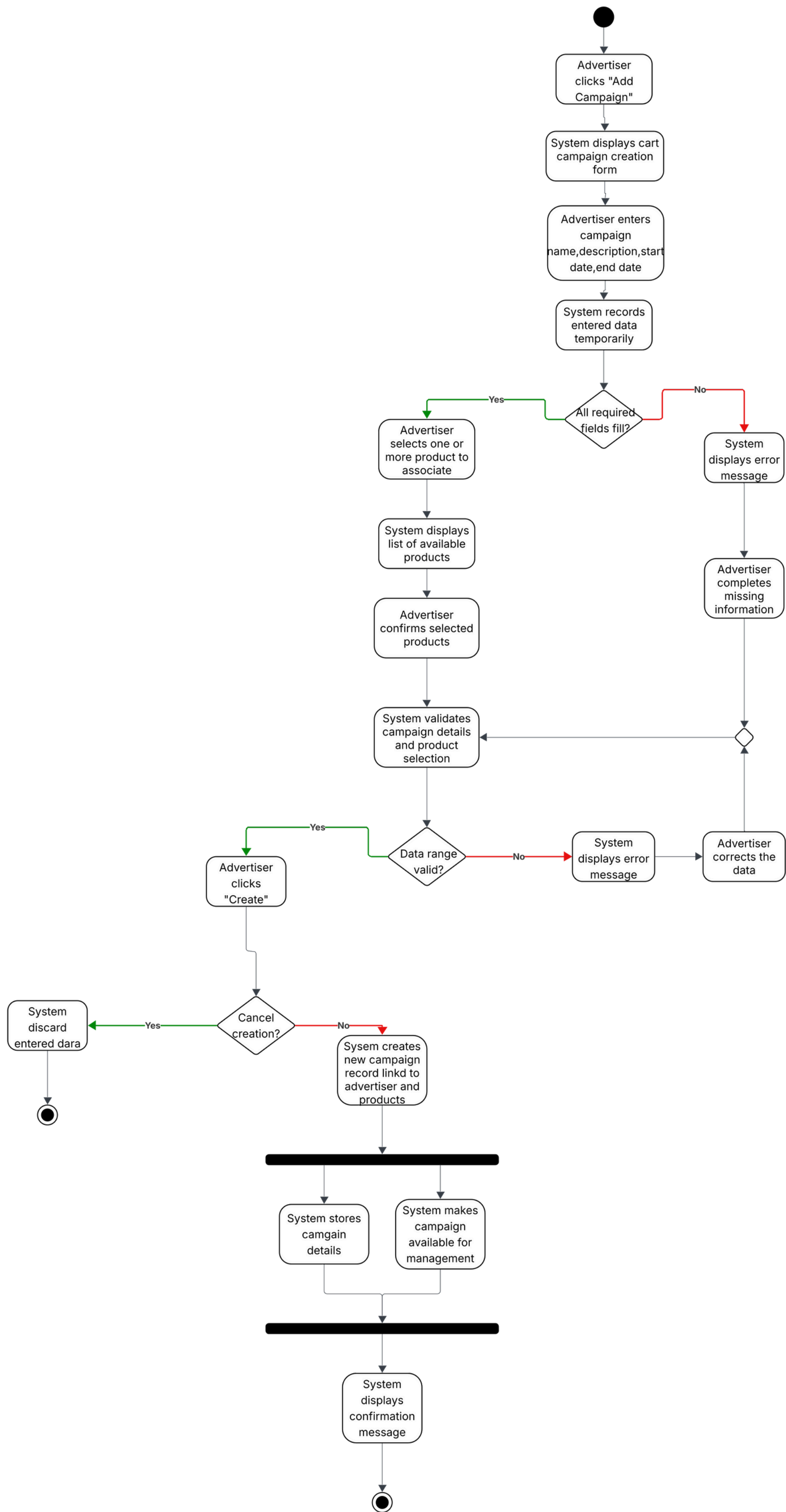
Campaign creation cancelled

Alternative scenario: Campaign creation canceled

- 9A1. The advertiser decides to cancel before confirming campaign creation.
- 9A2. The system discards all entered data.
- 9A3. The use case ends.

Post-condition:

- 1.The system creates a new campaign record linked to the advertiser and the selected products.
- 2.The system stores all campaign details, including name, description, and duration.
- 3.The campaign becomes available for further management or activation by the advertiser.
- 4.If the advertiser cancels the creation, no campaign record is created.



Use Case Scenario "Purchase Cart"

Actor: Shopper

Purpose and context: A shopper adds needed items to the cart and after completing that proceeds to payment and delivery options.

Assumption: 1. The user is satisfied with the selected cart items also with their number, prices

Precondition: 1. The user is viewing the cart page with the list of selected items for PURCHASE.
2. There should be at least one item in the cart

Basic flow of events:

1. The shopper clicks on "Proceed to payment" button
2. "Select delivery address" page
 - a. the shopper already has an information of delivery address in the user account and selects it
 - b. In the other case the shopper fills the form correctly in the page of delivery address
3. Then payment page is shown , where the shopper selects a preferred payment method (not by cash)
4. After selecting that payment method generates a link for a transaction
5. After a successful purchase the message about it is displayed,**the check is sent to the email** and a link to track the order status,also it clears the cart of the shopper

Alternative flow of events:

Product in the cart are unavailable

2b1. The system displays the list of such products and return to the main page

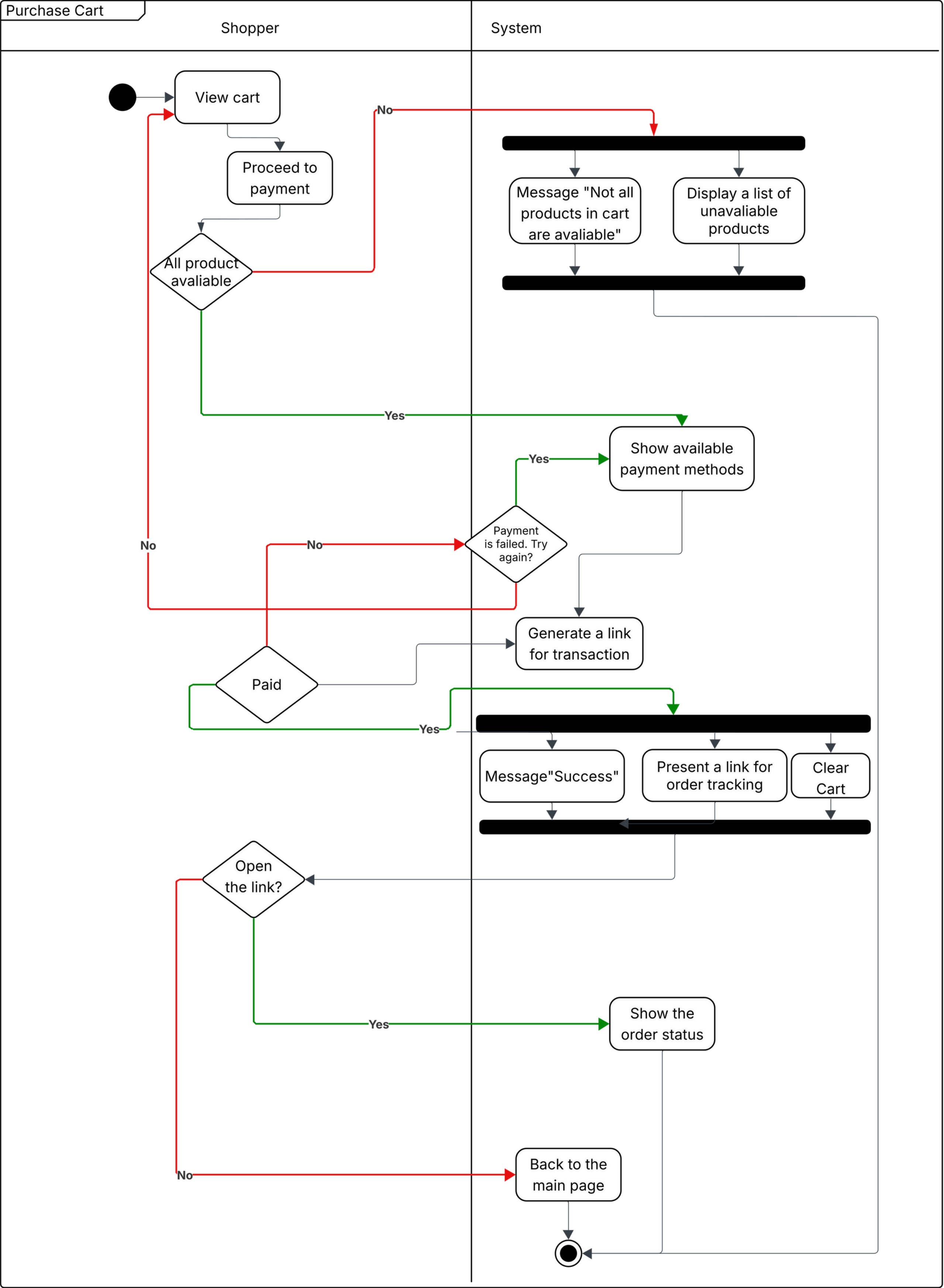
The payment is failed

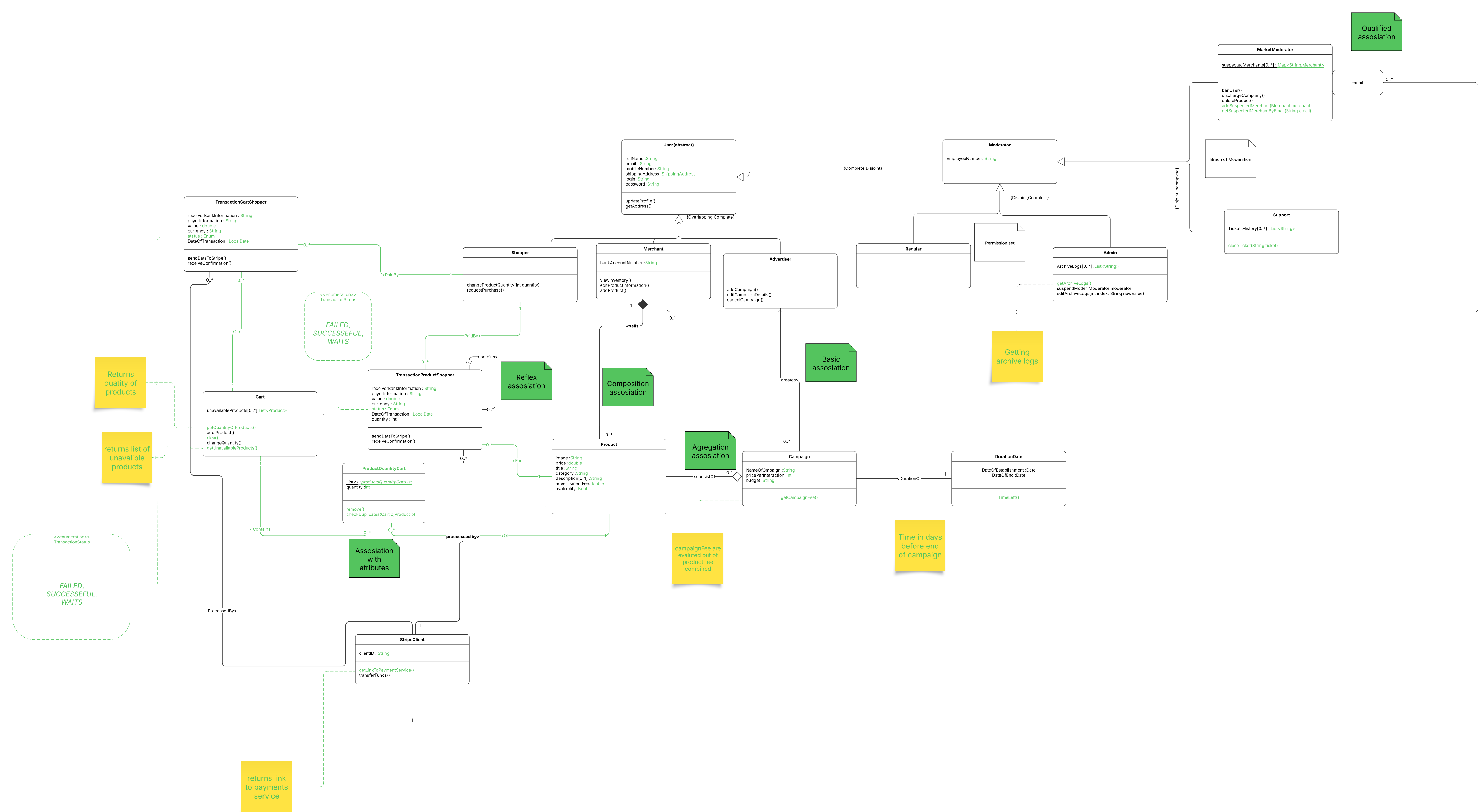
4a1. The system displays the info and a button "Retry", the transaction link is regenerated

Post condition:

Log info: Update on history of transactions

Stock: Some items will be reduced or even out of stock





Dynamic Analysis on "Add Product","Add campaign","Purchase Cart" use cases & "Cart class" state diagram.

Following the above-presented diagrams, a dynamic analysis was performed. The result of this analysis indicated a need for further expansion on the design class diagram to encompass newly identified requirements and behaviors of the system. All introduced changes can be seen in the below presented final design class diagram.

"Cart class" state diagram.

- 1.First of all transaction might be failed so we need to add a status of transaction in transaction class
- 2.Card is cleared every time shopper use purchaseCart() method seccussfully so we need to add a method to clear the card so the shopper can cleared it if he wanted without buying the whole thing it might be a good method to make a money but not quite handy and fair.
- 3.Since in our card might be unavaliable products we need to show and evaluate which products are unvaliable in the card in do it dinamicly so we gonna add filed like /unavalibleProducts do display it to the shopper in case he want buy a cart and his cart contain such unavalible products

"Add Product"

No comments its gonna work with implementation we have now at least i hope its gonna work.

"Add campaign"

- 1.Campaign now have validation date and date of decloration and date of end which is logical but with simple implementation that is unachivable we need to add class Duration class for assosiating it with the campaign since its gonna have field like dateOfDecloration,dateOfEnd or smth similar and evaluating the gap between to know then campaign needed to be deleted

"Purchase Cart"

- 1 Have pretty much same problems as Cart class in general and they already described

/UnavalibaleProduct-its list of products which is curently in cart and have unvaible status.

Method clear() - clears out the cart out of products

Status in transaction is reffers to is transaction failed seccesful or waiting to be paid

DurationDate is new class explicetly for campaign its contain startDate and EndDate of the campaign also it evalute dimanicly how much time this campaign left in the future might be used for other implementations

The presented class diagram illustrates the changes necessary to model UML constructs that do not exist in the Java language. The changes introduced are highlighted in green. The following section will describe the transformation process of certain UML constructs such as attributes types.

Runtime validation logic:

Using the java custom runtime exception and basic if checks we making validation we could use Java Bean validation for same purposes and make it faster and simple and more like a c#.net but we dont what u goona do about it

Optional Attributes:

Attributes that may contain no value they are represented in UML as attribute with [0..1] notation and implemented as null values in java with separate constructor so if dont wanna create one u just dont type on while constacting the object examples: Product.description[0..1]:String

Multi-value attributes:

Java is nativly supporting Multi-value attributes as classes inherited from Collection class we using for our purposes ArrayList<E> from java.util this its worth mentioning how and what for we using that collection first of all to store in our customers cart a peroduct that he added there and potentially willing to buy we used arraylist generic nature to fill our needs in different scenarios we save custom class products in our card using that implementation as well as we using more in basic way in our moderator system which can write and adjust tickets examples:Car-unavailableProducts[0..*]:List<Product>,Admin-ArchivesLog[0..*]:List<String>,MarketModerator-suspectedMerchants[0..*]:List<String>,Map<String,Merchant>,Support-TicketHistory[0..*]:List<String>

Derived Attributes

Derived attributes its such attributes which is not located as field in our implementation and rather count on the run in java we used regular get methods as in encapsulation principle to simulate existing of these attributes for example:timeLeft so public int TimeLeft() { return dateOfExpiration.getDays() - dateOfEstablishment.getDays(); }

Enums(tagged values)

We use it Transaction state in transaction class to differentiate from one another and base behaviour of our system based on it we use enums or enumerators for that purposes which is represented in java based library enum Status implements Serializable{ FAILED, SUCCESSFUL, WAITS }

public class Transaction implements Serializable{

Complex attributes(class attributes)

Complex attributes its such attributes that contain more than one simple field or could have more complex logic in their behaviour compare to the basic attributes and field we implemented it using standard java tools they represented as inner classes of the class which have such complex attribute as an example i have here Date class which is inner class of DurationDate and it uses two fields Date one called start of second called end and they represent day month hour and year public static class Date implements Serializable { private int year; private int month; private int day; private int hour;

Date(int year, int month, int day, int hour) { if (month < 1 || month > 12 || day < 1 || day > 31 || hour < 0 || hour > 23) { throw new IllegalArgumentException("Invalid date format"); } this.year = year; this.month = month; this.day = day; this.hour = hour; }

public class DurationDate implements Serializable { private static final List<DurationDate> extent = new ArrayList<>(); private static final String EXT_FILE = "duration_exten.ser"; private Date dateOfEstablishment; private Date dateOfExpiration;

Extension classes(how we saving our progress):

Since using databases is forbidden and some mysterious whispers from within the reality dictate us to use an extension class to save our systems basically it method of saving objects in the file and initialize it later in even completely another session of the programs and what was i describing was a Serialization and particularly what we using in java for such purposes is Serializable interface which of our classes inherit from to get that functionality also we have a static field which is represented with object of ArrayList which as we remember is a collection and everytime constructor of the class are triggered ArrayList get filled up with our class and every point of them system all objects of the specific class can be saved in through a static method also we using inheritance because by making that functionality in User all of its child classes like Merchant, Advertiser, Shopper already have it implementation but its formality we can from any point we wont use any additional frameworks all of this just basic java so that it is how it is

Tests:

Here actually we using the additional functionality to make test we using JUnit testing for java as tool to create test easier and faster and more be more precise in our testing so we test our i would called straight forward normal people called dumb validation of the attributes for every class we implemented and as well test testing how our class serialization work or how our Extension classes system work

General association implementation

AddMethod(): In most of our relations the direction is reversed, so the add method of one object calls the add method of another object. To avoid looping, every connected add method has an if-statement that checks whether the relation already exists in the container. If it does, the process stops.

RemoveMethod(): Basically the same but in reverse. The removal process can be triggered from either side, and it automatically removes the bound in the opposite object. This is controlled with an if-statement check.

EditMethod(): We don't have many examples of this, but it works in reverse in our case. For example, a Product cannot modify itself, but a Merchant can modify it through setters in the Product class. These setters check whether the Product belongs to the Merchant performing the modification.

Composition Association: Represented in our project as Merchant and Products. Products cannot exist without the Merchant who sells them. Since we are implementing everything in Java, we add a method to delete a Merchant. It goes through the list of associations and deletes all bounds in both the Merchant and the Products created by it. Because Java does not allow manual deletion of objects, the idea is that without references the garbage collector should delete them, but I am not fully sure about that.

Qualified Association: Implemented as a bound between a Supervisor (MarketModerator object) and a Merchant in the list of suspected merchants. It takes the form of a map where the key is the email and the value is the Merchant object. Since the Merchant map is static, any moderator can access it, but each Merchant also keeps a record of which moderator added them to the suspected container. Since a Merchant can change its email at any time, the map updates to the Merchant's new email. That is basically how it works.

Association Class: Represented as ProductQuantityCard. Every time a Product is added to the Cart, it uses the constructor of that class, which requires a Cart object, a Product object, and an integer quantity. It can be created from both the Product and the Cart, which is not the most logical thing, but it works in reverse and functions properly. It also has a unique feature I implemented during a sleepless night: instead of throwing an identity exception, it checks if the same Product was added to the same Cart before, regardless of quantity. If it was, it adjusts the existing relation and rewrites the quantity to be the sum of the old and new values. I thought it was cool, even if nobody will read the description or code.

Reflex Association: As an example of reflex association, we use the Transaction class. It has a child field and a parent field, and the addChild method is reversed because it all comes from the same class, and both objects can use it. It is more logical to add children to the original object rather than the other way around, but still, when you add a child transaction, the child adds you back as the parent. The difference is that the child is a container, and the parent is a single field represented in the object. It is three in the morning and I am writing this on the go, so I might have misinterpreted something.

Association Validation: Done with a basic if-check to see if the container already holds the relation, which breaks the loop. As mentioned above, the association class also performs an individuality check, but instead of stopping the method or firing an exception, it adds the new quantity to the original one. Since there is no way to stop a constructor, it creates an empty object not tied to any relation, which is instantly removed by the Java garbage collector. Some associations do not need a check (e.g., reflex, qualified), but we still check for null objects. Reverse connections are often done through recursion, where the add method of one class calls the add method of the other and vice versa. I already explained how we prevent endless looping. To sum up our implementation: we used the provided documents and tried to keep everything as simple and functional as possible.